

103. Huit palettes, et teindre chaque case

Le ciel en bleu, le mur en brique, l'herbe en vert : chaque case de l'écran choisit sa palette.

1. Ce qu'il faut comprendre

Une palette n'a que **quatre** teintes. Pour un ciel, un mur et de l'herbe, il en faut plus : la Game Boy Color en a **huit** pour le décor, numérotées de 0 à 7. Huit palettes de quatre : **32 couleurs** à l'écran en même temps pour le fond.

`teindre(colonne, ligne, palette)` dit quelle palette emploie **une case** de l'écran. Le choix est rangé dans une **seconde carte**, cachée derrière la première : pour chaque case, l'une dit **quelle tuile**, l'autre dit **quelle palette**. C'est le seul ajout de la couleur à la façon de dessiner — les tuiles, elles, ne changent pas.

La contrainte à retenir : **une case = une palette = quatre couleurs**. On ne peut pas mettre cinq couleurs dans une même tuile de 8×8 . Les graphistes découpent donc leurs dessins en tuiles qui tiennent chacune dans une palette.

Et une même tuile peut servir avec **des palettes différentes** : l'herbe et le buisson ci-dessous sont le même dessin, `TOUFFE`, en palette 2 puis en palette 3. Une tuile, deux objets.

Toutes ces écritures se font écran éteint (le VBlank, chapitre 3).

À toi : ajoute une rangée d'eau en palette 4, avec des bleus.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **830 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	393 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), teindre() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo

```
EffacerCarte      14 o ← démarrage — vide les deux cartes de l'écran
Tuiles           192 o ← CopierTuiles — 9 lettres sur 43, et 3 tuiles dessinées
"HUIT PALETTES"  13 o ← texte() — un texte gravé
```

Lettres gravées : AEHILPSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
couleurFond() × 12  144 o sur place, dans main()
teindre()     × 3   84 o sur place, dans main() + AdresseCase (10 o)
poser()       × 4   52 o sur place, dans main() + AdresseCase (10 o)
ecran()       × 2   21 o sur place, dans main() + AttendreVBlank (16 o)
texte()       × 1   11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy Color) : un paysage : un ciel, un mur de briques,
// une rangée d'herbe et de buissons, et encore des briques.
//
// Ce qui est nouveau : plusieurs palettes, et teindre(colonne, ligne, palette).
// Le fond a 8 palettes (0 à 7) de 4 couleurs chacune.
// Chaque case de l'écran choisit SA palette avec teindre().
// Règle : une case = une palette = 4 couleurs au plus.

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <couleurFond>    // choisit une couleur d'une palette du fond
#include <ecran>          // éteint ou rallume l'écran
#include <poser>          // pose une tuile sur une case du fond
#include <teindre>        // met une case du fond dans une palette
#include <texte>          // écrit un texte à l'écran
```

2. UN DESSIN — CIEL

```
Tuile CIEL = {
    "00000000", "00000000", "00000000", "00000000",
    "00000000", "00000000", "00000000", "00000000",
};
```



à l'écran

3. UN DESSIN — BRIQUE

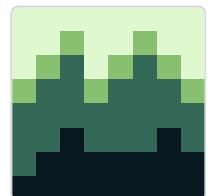
```
Tuile BRIQUE = {
    "33333333", "11131111", "11131111", "33333333",
    "13111113", "13111113", "33333333", "11131111",
};
```



à l'écran

4. UN DESSIN — TOUFFE

```
Tuile TOUFFE = {
    "00000000", "00100100", "01201210", "12212221",
    "22222222", "22322232", "23333333", "33333333",
};
```



à l'écran

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    // couleurFond(paLETTE, indice, rouge, vert, bleu), chaque lumière de 0 à 31.
    // Palette 0 : le ciel.
    couleurFond(0, 0, 18, 26, 31); // indice 0 : bleu ciel
    couleurFond(0, 3, 2, 4, 14); // indice 3 : bleu sombre (pour le texte)
    // Palette 1 : la brique.
    couleurFond(1, 1, 26, 12, 6); // indice 1 : brique
    couleurFond(1, 3, 10, 4, 2); // indice 3 : joints sombres
    // Palette 2 : l'herbe. Palette 3 : un buisson plus sombre.
    couleurFond(2, 0, 18, 26, 31); // le ciel derrière l'herbe
    couleurFond(2, 1, 20, 31, 10); // vert clair
    couleurFond(2, 2, 8, 24, 4); // vert
    couleurFond(2, 3, 2, 12, 2); // vert sombre
    couleurFond(3, 0, 18, 26, 31); // le ciel derrière le buisson
    couleurFond(3, 1, 10, 20, 16); // des verts plus froids et plus sombres
    couleurFond(3, 2, 4, 14, 8);
    couleurFond(3, 3, 0, 6, 4);

    ecran(0);                                // Dessin écran éteint.
    for (uint8_t c = 0; c < 20; c++) { // Pour chaque colonne c :
        for (uint8_t l = 0; l < 10; l++) poser(c, l, CIEL); // lignes 0 à 9 : le ciel, palette 0
        for (uint8_t l = 10; l < 14; l++) { // lignes 10 à 13 :
            poser(c, l, BRIQUE); // des briques,
            teindre(c, l, 1); // en palette 1.
        }
        poser(c, 14, TOUFFE); // ligne 14 : une touffe,
        teindre(c, 14, c < 10 ? 2 : 3); // le même dessin, deux palettes
        // « c < 10 ? 2 : 3 » se lit : SI c < 10 ALORS 2
        // SINON 3. Moitié gauche : herbe ; droite : buisson.
        for (uint8_t l = 15; l < 18; l++) { // lignes 15 à 17 :
            poser(c, l, BRIQUE); // encore des briques.
            teindre(c, l, 1);
        }
    }
    texte(4, 3, "HUIT PALETTES");
    ecran(1);                                // On rallume.

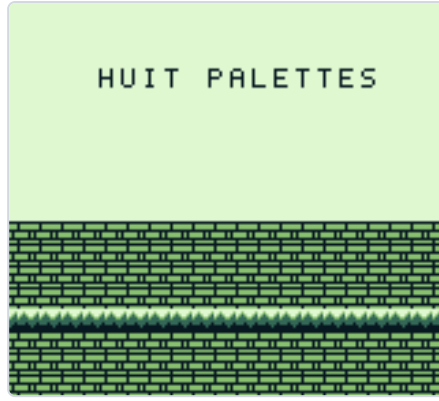
    while (true) {                            // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un ciel bleu clair, un mur de briques, et une bande d'herbe : verte à gauche, plus sombre à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le ciel prend la palette 0 (18/26/31)
- ✓ la brique prend la palette 1 (26/12/6)
- ✓ l'herbe, à gauche, la palette 2 (8/24/4)
- ✓ le même dessin à droite, la palette 3 (4/14/8)

Ensuite : **104. Les lutins en couleur** — chapitre 16 : La couleur : la Game Boy Color. · [gameboy3](#) — écrire une cartouche Game Boy en C++